

TRAINER REFERENCE



Programme Index and Continuity Map

Enterprise Data Engineering with SQL, Redis & Kafka

120 Hours | 12 Modules

What is in this kit

Every module folder contains the same five deliverables. Nothing is shared between folders except the SQL scripts in `00_Programme_Overview/sql`, which every module uses.

File	For	Release
Mxx_Presentation.pptx	The trainer, projected	During the session
Mxx_Course_Material.pdf	Learners, to keep	Start of the module
Mxx_Lab_Guide_with_Solutions.pdf	Both; expected output is inline	During the lab
Mxx_Exercises.pdf	Learners	After the lab
Mxx_Exercise_Solutions.pdf	Learners	After the practice window

The programme overview folder holds four documents that are not tied to a single module:

File	For	Release
M00_Student_Setup_Guide.pdf	Learners	A week before day one
M00_Glossary_of_Terms.pdf	Learners	Day one, kept for all 12 modules
M00_Course_Material.pdf	Both	Day one
M00_Programme_Index_and_Continuity_Map.pdf	The trainer	-

Core and stretch. Every exercise document marks a **core set** that fits the session time, with the remainder as stretch work. The core set alone covers every objective of the module. Tell learners on day one that finishing the core set is finishing the exercise, because fresh graduates who think they are behind stop asking questions.

Three modules also ship runnable code:

Module	Folder	Contains
7	Module_07_NoSQL_and_Redis/code	<code>icici_cache.py</code> - the Redis reference implementation
8	Module_08_Apache_Kafka/code	<code>docker-compose.yml</code> and <code>icici_events.py</code>
12	Module_12_Capstone_Banking_System/code	<code>icici_bts.py</code> - the complete capstone system

The full inventory

Module	Hours	Slides	Folder
Programme overview	-	-	00_Programme_Overview
1. Introduction to Databases and RDBMS Fundamentals	6	42	Module_01_RDBMS_Fundamentals
2. SQL Fundamentals, DDL and DML	8	46	Module_02_SQL_Fundamentals_DDL_DML
3. Advanced SQL Queries	8	46	Module_03_Advanced_SQL_Queries
4. Constraints, Design and Normalization	8	43	Module_04_Database_Design_and_Normalization
5. Query Optimization and Database Programming	8	48	Module_05_Query_Optimization_and_Programming
6. Transactions and Advanced SQL	6	40	Module_06_Transactions_and_Advanced_SQL
7. NoSQL and Redis	10	41	Module_07_NoSQL_and_Redis
8. Apache Kafka	10	38	Module_08_Apache_Kafka
9. Database Security and Enterprise Integration	6	38	Module_09_Security_and_Enterprise_Integration
10. Microsoft SQL Server Administration	20	57	Module_10_SQL_Server_Administration
11. Enterprise Data Migration	20	45	Module_11_Enterprise_Data_Migration
12. Capstone Project	10	28	Module_12_Capstone_Banking_System

Total: 120 hours.

The continuity map

The programme is one continuous story about one bank. Each module opens with a support ticket that the previous module's work made visible, and closes by handing something concrete to the next.

The ticket thread

Module	Ticket	The problem
1	ICICI-4471	A confirmed transfer vanished after a server restart
2	ICICI-4498	A fixed-deposit table built with no constraints, full of bad data
3	ICICI-4530	Branch managers rebuilding the same report in Excel every month
4	ICICI-4577	One branch spelled four different ways in the customer master
5	ICICI-4612	The mini-statement takes nine seconds
6	ICICI-4655	Two ATMs both approved a withdrawal against one balance
7	ICICI-4701	Balance enquiry is 60 percent of core banking load
8	ICICI-4755	An SMS gateway outage stopped fund transfers
9	ICICI-4802	Six findings from the annual database audit
10	ICICI-4850	The reporting warehouse has no owner
11	ICICI-4900	Retire the legacy Oracle core banking system
12	ICICI-5000	Build and demonstrate the whole thing

What each module hands forward

Module	Hands to the next
1	The database, the ACID vocabulary, and the durability incident that motivates everything
2	The tables, the constraints, and <code>fixed_deposit</code> built correctly
3	Joins, aggregation and the branch performance pack
4	A normalized schema, the ER model, and referential integrity
5	Indexes, execution plans, <code>sp_fund_transfer</code> and the function library
6	Correct concurrency, and the running-balance statement
7	The cache layer, and Pub/Sub losing a message nobody heard
8	The event backbone, the outbox, and idempotent consumers
9	Roles, backup, replication, monitoring, and the operating model
10	The SQL Server warehouse, which becomes Module 11's migration target
11	The migrated database the capstone is built on
12	-

Techniques that reappear

The point of the thread is that a technique taught once is used again, in a context where the learner did not expect it.

Taught in	Reused in
ACID durability (1)	Write-behind caching rejected (7); the outbox (8); RPO and RTO (9)
PRIMARY KEY (2)	Idempotent consumers (8); the <code>event_processed</code> table (12)
Delete order and foreign keys (4)	Migration load order (11)
EXCEPT (3)	Migration reconciliation (11); report validation (10)
Partial index (5)	The outbox's unpublished index (8)
FOR UPDATE (6)	WITH (UPDLOCK) in T-SQL (10); the capstone's concurrency proof (12)
SKIP LOCKED (6)	The outbox relay (8, 12)
Window functions (6)	Analytical reporting (10)
Cache invalidation (7)	Change data capture (8, 11)
Consumer lag (8)	Monitoring (9); the capstone health screen (12)
Staging then transform (10)	Migration ETL (11)
The four spellings of Koramangala (4)	SSIS lookup failures (10); migration lookup errors (11)

Banking objects that persist

The same 30 customers, 36 accounts, 8 branches and 495 transactions appear in every module. Account 9003 is used for most single-account examples, and accounts 9015, 9020 and 9021 are reserved for the concurrency demonstrations.

Object	Introduced	Used again in
branch, customer, account, account_txn	1	Every module
fixed_deposit	2	4, 5, 10
employee with manager_id	1	6 (recursive org chart), 11 (self-referencing load)
loan, loan_payment	1	3, 5, 10
branch_customer_flat	4	10, 11
outbox	8	9, 11, 12
event_processed	8	12

Trainer quick reference

Before day one

	Task
[]	PostgreSQL 18 installed, <code>icicibank</code> created and seeded
[]	PATH fixed on every machine , verified in a fresh window with <code>Get-Command psql</code> , <code>createdb</code>
[]	Oracle XE and MySQL installed for Module 1's comparison, and Module 11's source
[]	Docker Desktop working, for Redis and Kafka
[]	SQL Server Developer and SSMS installed before Module 10
[]	Python 3.11 with <code>psycopg</code> , <code>redis</code> and <code>confluent-kafka</code>
[]	Row counts verified: branch 8, customer 30, account 36, account_txn 495, employee 24, loan 12, loan_payment 144, card 32, beneficiary 60
[]	<code>M00_Student_Setup_Guide.pdf</code> sent to the batch a week ahead

The PATH, and why it is on this list

The PostgreSQL installer does **not** add its `bin` folder to the Windows PATH. Every command line tool lives there, so when `psql` is missing, `createdb`, `pg_dump`, `pg_restore` and `pg_basebackup` are all missing with it. Left unfixed it resurfaces in Module 5, Module 9 and Module 11, each time looking like a different problem.

Fix it once on day one, on every machine in the room:

1. System Properties, Environment Variables, Path, New.
2. Add `C:\Program Files\PostgreSQL\18\bin`.
3. **Open a new PowerShell window.** A window that was already open keeps the old PATH for its whole life, which is what makes learners think the fix did not work.

Then have every learner run this and show you two paths, not two errors:

```
Get-Command psql, createdb
```

Do this as a room, not one desk at a time. It takes four minutes together and roughly forty minutes if you let it surface individually across the morning.

Per module

1. **Open with the ticket.** Five minutes. Fresh graduates retain syntax far better when it solves a problem they can picture.
2. **Teach from the deck**, running each code slide live against the seeded database.
3. **Run the lab.** The guide is keystroke by keystroke with expected output after each step.
4. **Set the exercises.** Hand out only `Exercises.pdf`.
5. **Debrief with the solutions**, which explain why rather than what, and name the mistakes batches actually make.

Resetting between modules

Modules 2, 4, 9 and 11 deliberately modify the schema.

```
psql -U postgres -d icicibank -f 00_Programme_Overview/sql/01_schema_postgres.sql
psql -U postgres -d icicibank -f 00_Programme_Overview/sql/02_seed_postgres.sql
```

Under a minute, and it restores a clean bank. **Tell learners this on day one** so nobody is afraid to experiment.

The modules most often underestimated

Module	Why
4	Normalization is conceptual; learners need the flat table in front of them to see the problem
6	Needs two terminals per learner, and the setup eats the session if unprepared
8	Kafka setup consumes more time than planned. Start the containers during the break before
10	Twenty hours and the largest topic list. Do not attempt it in one day
12	Teams gold-plate the schema and arrive with nothing to demonstrate. Intervene at hour four

The demonstrations worth rehearsing yourself

Module	Demonstration
1	Durability: kill the server mid-transaction
6	Two sessions, one balance, the lost update and its three fixes
7	The stale cache: 44 database calls become 1
8	Stop a consumer, make transfers, restart it, watch it catch up
9	Point-in-time recovery to the instant before a mistaken DELETE
10	25,418 logical reads becoming 5
11	The Oracle DATE losing its time, caught only by row-level reconciliation

Each of these is the moment the module's idea becomes concrete. They are worth running once alone before running them in front of a room.